ソフトウェア開発 / 利用者ガイド

ArquitecturaRepo

完全ユーザーマニュアル

バージョン 1.4 / 日本語

全8章：導入、インストール、導入設定、日常運用、変更依頼、復旧、受け入れ、実践的なクラブ管理の例。

MATÍAS ACUÑA · 2026年11月

ArquitecturaRepoの公開ソースに照合した利用者向け文書。

目次

00 — はじめに 5
この章で学ぶこと 5
1. 導入されるものと導入されないもの 5
2. 通常のコーディングチャットとの違い 6
3. 利用者と各担当者の責任 7
4. 証拠、現在性、履歴の扱い 8
5. 要求・製品状態・実装の三層構造 9
6. 小さく始めるための考え方 10
7. 八章の読み進め方 10
8. 利用者が決めるべき事項 11
9. 会員管理を例とした全体像 11
10. この仕組みの効果を判断する方法 12
参照資料と適用範囲 13

01 — インストールと初期設定 14
この章で学ぶこと 14
1. インストールに必要な環境と権限 14
2. スターターとアプリケーションは別のもの 15
3. 導入前に対象を確認する 16
4. 六つのプロファイルを選ぶ 16
5. 新規プロジェクトに管理層を生成する 17
6. 導入結果を検証する 18
7. 既存の管理済みプロジェクトには別の手順がある . . . 19
8. 失敗した場合の判断 19
参照元と適用範囲 20

02 — Plannerによる発見と正式な採用 21
この章で学ぶこと 21
1. チャットは製品の正本ではない 21
2. 作業領域を読み取り専用で準備する 22
3. Plannerは製品を変える質問を優先する 23
4. 要求、Product State、コードを一緒に見る 23
5. 採用とは現実へ接続すること 24

6. 新しい文脈で復元できるかを試す 25
7. 最初のキャンペーンを認可する 25
8. 会員管理システムの初回セッション 26
参照元と適用範囲 27

03 — 日常運用と継続的な実装 28
この章で学ぶこと 28
1. 昨日のチャットを覚えている必要はない 28
2. 願望を能力に変換する 29
3. I3は途中の失敗も引き受ける 29
4. 外部行為には境界を設ける 30
5. 完了報告は「PASS」という一語では足りない 30
6. Gitは継続性の材料であって権限ではない 31
7. 一日の終わりに答える五つの質問 32
8. 仮想クラブでの一日の流れ 32
参照元と適用範囲 33

04 — 変更を依頼する方法 34
この章で学ぶこと 34
1. 最初に「何を可能にしたいか」を伝える 34
2. 短い変更依頼のひな型 35
3. 利用者が決めるものと委任するもの 36
4. 意見を変えても履歴を壊さない 36
5. 品質に対する要求を明確にする 37
6. 優先順位は実際の価値で決める 37
7. 判断が本当に必要なとき 38
8. 良い依頼の具体例 38
参照元と適用範囲 39

05 — エラーの診断と安全な復旧 40
この章で学ぶこと 40
1. 原因より先に証拠を保護する 40
2. 環境とアカウントの問題 41
3. 管理層の検証が失敗する場合 42
4. Gitは安全な万能取り消しではない 42
5. 同じ失敗を何度も繰り返さない 43
6. セキュリティ、個人情報、ライセンス 43
7. 復旧の終了条件 44

8. 三つの復旧例 44
参照元と適用範囲 45

06 — 能力の受け入れと作業の終了 47
この章で学ぶこと 47
1. 「機能を作った」だけでは能力にならない 47
2. 終了を五つの要素で考える 48
3. 正確な候補を確認する 48
4. 証拠が足りないときは作業を区別する 49
5. 三つの最終判断 50
6. 終わりのない検査を避ける 50
7. 受け入れ記録は新しいチャットでも読めるようにする . 51
8. 会員登録の判断例 51
参照元と適用範囲 52

07 — 実践例：小さなクラブ管理アプリを構築する 53
この章で学ぶこと 53
1. 架空のクラブの問題を定義する 53
2. 仕様を決める質問をする 54
3. スターターを正しい場所へ導入する 55
4. 採用のための技術調査 55
5. 最初の能力：会員の登録と一覧 56
6. 第一候補で起きる二つの問題 57
7. 利用者の確認と受け入れ 57
8. 二つ目の能力：会費を記録する 58
9. 最後の復元演習 58
参照元と適用範囲 59

00 — はじめに

ArquitecturaRepo · 完全ユーザーマニュアル · v1.4版 · 日本語

> この章では、ArquitecturaRepoの目的、責任範囲、導入前に確認すべきことを説明します。公開済みのv1.4.0リリースに対応した解説ですが、特定のアプリケーションが完成・検証済みであることを意味しません。実際の導入コマンドは公開導入ガイドとリリースノートを優先してください。

この章で学ぶこと

1. 導入されるものと導入されないもの
2. 通常のコーディングチャットとの違い
3. 利用者と各担当者の責任
4. 証拠、現在性、履歴の扱い
5. 要求・製品状態・実装の三層構造
6. 小さく始めるための考え方
7. 八章の読み進め方
8. 利用者が決めるべき事項
9. 会員管理を例とした全体像
10. この仕組みの効果を判断する方法

1. 導入されるものと導入されないもの

ArquitecturaRepoは、ソフトウェア開発を継続可能な形で管理するための、リポジトリ中心の公開スターターです。責任と権限を定義するプロトコル、再利用可能なテンプレート、用途別の導入プロファイル、検証可能な状態の形式、Pythonによる検査・復旧ツールなどを備えています。必要に応じて、ローカルのキャンペーン制御機構を追加することもできます。

これらを導入しても、要件が自動的に確定したり、完成したアプリケーションが生成されたりするわけではありません。実際の

製品には、利用者の問題を理解すること、技術方式を選ぶこと、コードを書くこと、実環境で動かすこと、失敗条件を検証すること、そして必要な人による受け入れ判断が必要です。スターターは、その仕事の責任分担と証拠のつながりを整理するための土台です。

バージョンの境界も重要です。v1.4.0には、固定されたリリース参照、配布用ZIP、SHA-256チェックサム、マニフェストがあります。継続的に更新されるmainブランチには、リリース後に作成された詳細マニュアルが追加される場合があります。mainで公開された新しい文書が、過去に作成済みのZIPへさかのぼって含まれることはありません。再現性のある導入には、実際に取得した配布物の識別と照合が必要です。

スターター自身のテストが成功したことと、そのスターターを利用する製品が安全に動くことも別の主張です。たとえば、合成データを使った参照アプリケーションは、特定の管理機構が動くかを確かめるために役立ちます。しかし、別の企業の会員管理アプリが安全であるという証拠にはなりません。検証されていない事項は、完了と書くのではなく「不明」「未検証」「HOLD」などとして残します。

特定のAIモデル、外部API、クラウドサービスへの契約は必須ではありません。人間だけのチームでも原則を適用できます。プログラミングエージェントを使う場合も、その役割、許可範囲、費用上限、実行環境を別に確認します。ツールが接続されていることは、支払い、公開、実データの送信を許可したことにはなりません。

2. 通常のコーディングチャットとの違い

短い実験では、一つのチャットに「この機能を作って」と依頼し、返ってきたコードを試す方法で十分なことがあります。対話による開発は速く、アイデアを確かめるのにも適しています。しかし、数週間続く開発や複数の担当者が関わる開発では、会話だけに現在の事実を保存すると問題が起こります。

新しい会話は、前の会話のすべてを覚えているとは限りません。以前「PASS」と報告されたテストが、現在のコードに対して実行

されたものかどうかも不明になりがちです。顧客が変更したはずの要求が古い説明に埋もれることもあります。結果として、依頼者が何度も技術的な経緯を説明し、ブランチやデータベースの整合性まで管理することになります。

ArquitecturaRepoが重視するのは、この依存を減らすことです。現在の要求、責任者、認可済みの作業範囲、コードの正確な参照、検証結果、未解決の問題を、リポジトリ内の適切な正本に残します。次の担当者がチャットの全文を読まずに、許される次の行動を再構成できることを目指します。

ただし、常にチャット単独より優れていると断言することはできません。小さな一回限りのスクリプトに対して、複雑な管理層を作るのは過剰かもしれません。一方、データを保持し、担当者が交代し、長期的な変更がある製品では、正しい引き継ぎと復旧が大きな価値を持ちます。比較すべきものは文書の量ではなく、完成した機能、修正に要する時間、誤認の減少、引き継ぎの確実さです。

同じ小規模な課題で試すと違いが見えます。最初のコードを作るまでの時間だけでなく、要求を後から変えたときに正しく追従できるか、新しい担当者が安全に作業を続けられるか、実際に受け入れられた版を特定できるかを比較してください。

3. 利用者と各担当者の責任

CLIENTは製品の意図を持つ依頼者です。誰のどの問題を解決するか、どの機能を優先するか、どのような操作感が必要か、どの費用や外部行為を認めるかを決定します。UIが技術的に動いていても使いにくければ、依頼者は改善を求められます。一方、Gitの衝突解消やデータベース移行の内部設計を毎回決める必要はありません。

内部の役割は四種類ですが、必ず四人の人間や四つのAIモデルが必要という意味ではありません。

役割	主な責任	避けるべきこと
I1 Master Owner（必要な場合）	複数製品・共有資源・全体方針の整合性	個別バグをすべて直接計画すること
I2 Planner	要求の解釈、技術戦略、能力単位の計画、戦略的受け入れ	単体テスト失敗ごとに新キャンペーンを作ること
I3 Execution Controller	実装、統合、デバッグ、再試行、復旧、レビュー調整	新しい製品要求を勝手に決めること
I4 専門担当・レビュアー	限定された実装、調査、検証、独立レビュー	依頼者の判断や最終権限を代行すること

小さな案件では一人が複数の内部役割を担ってもかまいません。ただし、利害の衝突がある重要な検証では、実装者から独立した確認が必要です。別の名前のチャットを開くだけでは、独立性が保証されるわけではありません。

Plannerが能力の目的と境界を確定し、キャンペーンが認可された後は、I3が日常的な技術作業を継続して担当します。通常の不具合、テストの失敗、内部リファクタリング、同一範囲での再試行は、原則として元のキャンペーンで解決します。逆に、ローカルアプリを公開インターネットサービスへ変更するような要求は、製品と安全性の条件を大きく変えるため、新しい判断が必要です。

4. 証拠、現在性、履歴の扱い

技術的な事実は、実際に存在するコード、設定、データベース、実行結果、成果物から確認します。CURRENTと名付けられたファイルであっても、古いコードを指していれば現在の事実とは限りません。過去の報告書は判断の経緯を知る手掛かりになりますが、最新の物理的証拠に優先しません。

製品意図については、依頼者の新しい明示的な決定が以前の要求を置き換えることがあります。その場合、現在の要求へ正しく反

映し、以前の決定が履歴であることを明確にします。単に最新のチャットが技術的事実を主張したという理由で、既存の正本を無効にしてはいけません。

証拠が存在しないときに「PASS」と記すのは危険です。未知のデータベースバージョン、確認されていない権限、実行していないテストは、そのまま未確認として扱います。重要な情報同士が矛盾していれば、その情報に依存する変更は一時停止し、読み取り専用の調査によって原因を確かめます。

導入直後の .agentic-sdlc/ が HOLD / UNTRUSTED_CONTEXT と表示するのは、正常な保護動作です。サンプルに含まれる仮の製品情報を、本物の要求として認めていないことを表します。必要な事実を調査し、依頼者の確認を得て、一貫した状態として正式に取り込んでから実装へ進みます。

5. 要求・製品状態・実装の三層構造

作業の意味を整理するため、三つの層で考えると便利です。L0は依頼者の発言と確認された決定です。L1はその内容を整理した現在の要求、製品モデル、Product Stateです。L2はコード、DB、キャンペーン、テスト、実際の実行結果です。

L1で「会員の支払い履歴を残す」と定義されていても、L2に保存処理が実装されていなければ能力は存在しません。また、L2でデータを保存できても、退会時に履歴まで削除するならL0の意図と一致しません。構文的に正しいプログラムと、正しい製品の完成は別です。

この区別は過剰な成功宣言を防ぎます。「支払いを扱える」という表現だけでは、架空の画面表示なのか、ローカルな記録なのか、実際の決済送信なのか分かりません。三つは異なる責任・安全条件・受け入れ基準を持ちます。正本には、何が実際に実装され、何が未承認なのかを明示します。

新しい担当者が三層の関係を追えることが、再構成可能なプロジェクトの基本です。チャットの要約だけではL2の現在性を保証できず、コードだけではL0の製品意図を完全には表せません。

6. 小さく始めるための考え方

最初から大規模な役割構成を用意する必要はありません。実際の危険に見合った最小プロファイルを選びます。小さな実験ではSMALL、長期間保持するデータや移行が必要な場合はSTATEFULを検討します。複数の開発領域や製品、重い成果物、高い影響を持つ処理では別のプロファイルが適しています。

まず短い製品メモを用意してください。利用者は誰か、最も重要な操作は何か、扱うデータは何か、どこで動くか、初版では何をしないか、未決定事項は何か。それだけでPlannerは意味のある調査を始められます。技術スタックやクラス設計まで依頼者が決める必要はありません。

その後、実際のリポジトリを読み取り専用で確認し、適切なコード・データの基準点を定めます。最初の納品は、全体を偽装した巨大なデモより、小さくても本当に動き、試せる能力が望ましいでしょう。詳細な導入手順は次章で扱います。

必要以上に難しい管理階層を導入しても、成果は増えません。ArquitecturaRepoの価値は、責任を複雑にすることではなく、必要な責任が曖昧にならないようにすることです。

7. 八章の読み進め方

章	主題	確認したい結果
00 はじめに	目的と責任	仕組みの限界を理解できる
01 インストール	配布物と初期設定	正しい場所へ安全に導入できる
02 Plannerと導入	調査と要求確認	新しい担当者が状態を再構成できる
03 日常運用	実装と継続	元の認可範囲で修正・検証を続けられる
04 変更依頼	製品の希望	必要な結果を技術詳細なしで伝えられる

章	主題	確認したい結果
05 故障と復旧	証拠と原因	データを守りつつ安全に回復できる
06 能力の受け入れ	候補と各ゲート	本当に検証された結果だけを承認できる
07 クラブ管理の例	一連の実践	発見から受け入れまで追跡できる

日本語ミニガイドとFAQは短い入門資料です。本書の代わりとなる60ページ級の翻訳として扱わないでください。旧スペイン語v1.3の完全マニュアルは歴史資料として別に保存されています。

8. 利用者が決めるべき事項

例として、地域のスポーツクラブが会員と会費を管理する小さなアプリを必要としているとします。依頼者は「一人の事務担当がローカルPCで使う。会員を検索でき、支払い履歴を残したい。初版ではオンライン決済をしない」と説明します。この情報から技術側は最初の能力を計画できます。

依頼者は、退会後の履歴を残すか、誰に閲覧権限があるか、どの操作が重要かを決めます。一方、SQLiteか別のDBを使うか、エラー処理をどのクラスで行うか、Gitブランチを何本用意するかは、通常は技術側の責任です。

未決定のことは未決定と書いてかまいません。たとえば「バックアップの頻度はまだ決めていない」と記録し、失う可能性のあるデータや運用負担をPlannerに説明してもらいます。無断の既定値を採用するより、正しい意思決定につながります。

9. 会員管理を例とした全体像

クラブの最初の要望は「会員を登録して検索すること」です。Plannerは要求だけでなく既存のコードやDBを確認します。会員

番号の一意性、同姓同名、退会後の履歴など、結果を変える質問を整理し、依頼者に必要な部分だけ確認します。

次に、最初の能力の範囲と検証基準を決めます。I3はコード・データ・画面を統合し、テストを実行します。同じ人が二回登録される不具合が見つかった場合、それは通常元のキャンペーンで修正します。新しい戦略計画を毎回作り直す必要はありません。

完成候補ができたら、単体テストだけでなく、データ保存、再起動後の検索、権限、実際の画面操作を確認します。利用者が操作を理解できるかという評価は、技術的な検査とは別に必要です。受け入れは、具体的な候補とその証拠に結び付けて記録します。

次に会費機能を作る場合、すでに検証済みの会員IDを利用し、重複請求や退会後の履歴を考慮します。前の能力が実際に使えることが次の能力の前提になります。第07章ではこの連鎖を詳しく追います。

10. この仕組みの効果を判断する方法

数回の開発サイクルの後で、次の点を確認してください。どの能力を実際に使えるようになったか。その候補はどこにあるか。何がテストされ、何がまだ不明か。具体的にどのリスクが減ったか。新しい担当者は、過去の会話を読まずに安全に続けられるか。

コミット数、文書数、エージェント数、実行時間は、それ自体では成果を示しません。小さなデータ整合性の修正が、たくさんの形式的報告より重要なこともあります。逆に、管理の仕組みが日常作業を必要以上に遅らせるなら、構成が過剰でないか見直すべきです。

ArquitecturaRepoは判断を自動化する魔法の製品ではありません。明確な責任、信頼できる証拠、復旧可能な状態を維持するための道具です。使う人が製品の意図に集中し、技術側が継続と整合性を責任を持って管理できるなら、導入には具体的な意味があります。

次へ：第01章「インストールと初期設定」。

参照資料と適用範囲

- v1.4.0公開リリース
- v1.4導入ガイド
- リリースノート
- アーキテクチャ概要
- 権限モデル
- MITライセンス

本章のクラブは架空の例です。特定の利用者のアプリケーションについて、完成・安全・公開の承認を主張するものではありません。

01 — インストールと初期設定

ArquitecturaRepo · 完全ユーザーマニュアル · v1.4版 · 日本語

> この章は、公開スターターのファイルを実際のアプリケーションへ安全に導入するための手順です。配布物の識別、対象ディレクトリ、初期HOLD、既存プロジェクトの保護を区別してください。最新の実行手順は公式の公開導入ガイドとリリースノートにあります。

この章で学ぶこと

1. インストールに必要な環境と権限
2. スターターと自分のアプリケーションの区別
3. 導入前の確認
4. 六つのプロファイル
5. 生成コマンドと初期状態
6. 検証と実際の採用との違い
7. 既存の管理層を持つプロジェクト
8. 安全なトラブルシューティング

1. インストールに必要な環境と権限

以下ではWindowsとPowerShellを例にします。Python、Git、ソースファイルへアクセスできるローカルディレクトリを用意してください。v1.4のポータブルCIではPython 3.12を使用しています。ただし特定のクラウド会社やコーディングモデルを契約する必要はありません。チャットやプログラミングエージェントは補助的な操作方法であり、実行対象と権限は別に確認します。

PowerShellで次のように現在の環境を確認できます。

```
& "python" --version
& "git" --version
```

```
Get-Location
```

コマンドが失敗した場合、端末のPATHとインタープリタを確認してください。必要なPythonパッケージは、配布物に明示された要件ファイルから導入します。無関係なインストーラを実行したり、エラーを隠すために検証コードを書き換えたりしないでください。導入の責任者は対象フォルダへ書き込める必要がありますが、外部公開、課金、機密情報の送信まで許可されているとは限りません。

最初の一歩で最も重要なのは、どのディレクトリを扱っているかという事実です。スターターのREADMEを開いたつもりで、自分のアプリケーションへ変更を加えることもあれば、反対にアプリケーションを変更したいのにスターターそのものを書き換えてしまう場合もあります。作業を始める前に、Gitリポジトリ名、現在のブランチ、未コミットの変更、書き込み可能な範囲を記録してください。

2. スターターとアプリケーションは別のもの

ArquitecturaRepoはプロトコル、プロファイル、テンプレート、検証ツールを配布する元のリポジトリです。一方、会員管理サービスや店舗向け予約アプリなどの実製品は、独立したアプリケーションリポジトリに置きます。自分用のスターターを保守する場合はforkを作成できますが、forkしただけで製品は作られません。

たとえば、次の構成は役割を分離しています。

```
C:\Proyectos\
    ArquitecturaRepo\
    club-app\
```

前者のフォルダからセットアップツールを実行し、後者のフォルダに管理層を導入します。スターターの合成サンプルを実製品の現在状態としてコピーしてはいけません。サンプル内の要求、所

有者、テスト成功、データベース参照は説明用であり、実際のアプリについて確認した事実ではありません。

取得したスターターの版も記録します。固定されたGitHub v1.4.0リリースには配布用ZIP、チェックサム、マニフェストがあります。現在のmainブランチで更新されたドキュメントが、以前のリリースZIPに自動的に含まれることはありません。元のコード、配布物、追加文書を混同しないことが、後の再現性とサポートの基本です。

3. 導入前に対象を確認する

スターター側から作業する前に、実際のアプリケーションを読み取り専用で点検します。Gitの状態とディレクトリが一致するか、すでにアプリのコードやデータが存在するか、以前のエージェントが作ったファイルがないかを調べます。既存の管理層 .agentic-sdlc/ がある場合は通常の新規生成を実行しません。

既存アプリにデータベース、移行履歴、稼働中のサービスがあるとき、初回導入と運用中の継承ではリスクが異なります。特に既存のProduct Stateをテンプレートで置き換えると、承認済みの決定と現在の技術証拠が失われます。その場合は既存の管理済みアプリケーション向け手順から始め、書き込みを伴わない事前点検を優先してください。

何もない新規製品でも、どのユーザーを想定するか、扱うデータ、初版の範囲、費用やライセンスの制約は簡単に書き出します。後続のPlannerが質問を絞り込みやすくなります。ただし、この時点では未回答の項目を勝手に決めたことにせず、不明事項として残します。

4. 六つのプロファイルを選ぶ

プロファイルは製品のリスクと運用形態で選びます。名称や条件の正本はprofiles/profiles.jsonです。

プロファイル	主な適用状況
SMALL	単純な製品で、重い状態管理や運用制御が不要
STATEFUL	永続データ、ユーザー、バックアップ、スキーマ変更が重要
ARTIFACT_HEAVY	多数の成果物、保存費用、由来、権利が重要
MULTI_WORKSTREAM	複数の開発領域が一つの製品を完成させる
MULTI_PRODUCT	複数製品が資源や運用方針を共有する
HIGH_CONSEQUENCE	プライバシー、安全性、資金、重大な不可逆操作を伴う

担当AIの人数を増やしたいという理由だけでMULTI_PRODUCTを選ぶ必要はありません。小さな試作であればSMALLを起点にできます。会員情報と会費を保持する実例ならSTATEFULを検討します。後で観測された要求が変化した場合は、プロファイル選択の根拠を更新してください。プロファイルがあること自体は、対象アプリケーションが安全であることの証明になりません。

5. 新規プロジェクトに管理層を生成する

対象を確認したら、スターターのルートから次のコマンドを実行します。これは新しいプロジェクトを例にしたコマンドであり、既存の管理層を持つアプリケーションにそのまま適用しないでください。

```
& "python" "tools\scaffold_project.py" --target "..
\mi-proyecto"    --profile    "SMALL"    --product-
id "MI_PROYECTO" --objective "Initial objective, awaiting c
-language "en" --no-feedback-workflow
```

会員管理アプリの想定例では、製品の状態を持つという事実に合わせて次のようにします。

```
& "python" "tools\scaffold_project.py" --target "..
\club-app"    --profile    "STATEFUL"    --product-
id "CLUB_MANAGER" --objective "Member and dues management f
-language "en" --no-feedback-workflow
```

パス、ID、目的は実例の値に置き換える必要があります。コマンドの言語指定は生成物の設定であり、この日本語版マニュアルの表示言語とは別です。現実の環境でサポートされる言語やオプションは、実行中のスクリプトのヘルプで確認します。

成功すると、対象のリポジトリに .agentic-sdlc/ が作成されます。通常のアプリケーションのソースを置き換えるコマンドではありません。初期状態が HOLD / UNTRUSTED_CONTEXT となっているのは、まだ実製品の要求、コード基準、権限、受け入れを確認していないからです。設定ファイルのステータスだけをREADYへ変えることは、導入成功の証明ではなく誤った状態の宣言です。

オプションの –with-runtime はローカルのキャンペーン制御に関する参考機構を加えますが、外部のAIモデルを導入したり、Codexのネイティブ実行を開始したり、課金を認めたりしません。–no-feedback-workflow は定期的なGitHub Actionsのスケジューラを無効化しますが、ローカルのメタデータレポート生成器とは別の設定です。

6. 導入結果を検証する

生成が成功したら、実製品のディレクトリで検証用依存関係と読み取り系のチェックを実行します。

```
& "python" -m pip install -r ".agentic-sdlc\requirements-
validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" -
-control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" -
-project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" -
-root ".agentic-sdlc"
```

project_statusは管理状態と不足事項を表示します。discover_projectは実際のファイルを調査し、reconstruct_contextは新しい担当者が現状を再構成できるか確かめます。これらがHOLDを返しても、未確認の製品情報を捏造していないという点で正しい場合があります。すべての構文チェックが成功しても、利用者の要求に沿ったアプリの能力を証明するわけではありません。

確認項目は三つです。正しいディレクトリに管理層が存在すること、既存のコードやワークフローが保存されていること、現在の未確認事項が読み取れることです。実際の製品についての探索、要件の承認、技術基準の採用は次章で扱います。インストール、採用、実装、受け入れの四つの結果は区別してください。

7. 既存の管理済みプロジェクトには別の手順がある

再度コマンドを実行する前に、.agentic-sdlc/ の内容を調べます。既存のCURRENT、権限記録、活動中のキャンペーン、実行リース、DBの参照、成果物の由来は復旧に必要な正本である可能性があります。強制削除や上書きは、標準的な問題解決方法にしてはいけません。

古いバージョンで構築したプロジェクトに新しい版を導入する場合も、互換性を憶測して移行しないでください。アップグレード条件と既存プロジェクトの安全な接続を確認し、移行対象と残すデータを明確にします。サンプルを生成し直すより、既存の基準点を尊重することが重要です。

8. 失敗した場合の判断

現象	起こり得る原因	安全な次の行動
Pythonが認識されない	インストールまたはPATHの問題	環境を修正し、コードや状態は変更しない

現象	起こり得る原因	安全な次の行動
管理層が見つからない	対象パスの違い、生成失敗	実行ログと作業ディレクトリを確認
既存の管理層がある	実製品がすでに管理されている	新規生成をやめ、読み取り専用の接続点検へ
jsonschemaがない	宣言済み依存が未導入	対応する要件ファイルから追加
UNTRUSTED_CONTEXT	実製品の事実がまだ採用されていない	Product Stateと要求の探索を続ける
コーディングツールが別のrepoを見る	作業領域の設定が異なる	書き込みを止め、対象と権限を確認
テストは緑だがアプリが動かない	管理機構の検査と実製品の実行は別	実際の起動・画面・データ操作を試す

作業後に「何を生成したか」「どこに生成したか」「何をまだ確認していないか」を記録してください。別の担当者がその三点を実データと照合できれば、次の採用作業に進むための出発点になります。失敗した検証を削除して成功したように見せる必要はありません。

次へ： 第02章では、Plannerによる発見、要件確認、正式な採用、最初の実装キャンペーンを扱います。

参照元と適用範囲

- 公開開始とサポート境界
- v1.4リリースノート
- フォークと初期設定
- 導入と移管
- 管理済みアプリケーションの事前点検
- 実行可能な検証
- プライバシーとフィードバック
- インストーラ実装

すべてのコマンドは説明用です。導入先で未確認の権限、環境、実行結果を証明するものではありません。

02 — Plannerによる発見と正式な採用

> 管理層が生成されたことと、その管理層が実際の製品を正しく表していることは別です。この章はHOLD / UNTRUSTED_CONTEXTから、検証可能な採用済みの基準点へ移るための利用者向け説明です。採用契約と製品発見の指示を併せて参照してください。

この章で学ぶこと

1. チャットの説明と製品の現状
2. 読み取り専用での技術調査
3. Plannerが確認する質問
4. 製品の要求と技術的基準点
5. 採用の権限と検証
6. 新しい文脈での再構成
7. 最初の認可されたキャンペーン
8. 会員管理システムでの実例

1. チャットは製品の正本ではない

最初の対話では、利用者は大きな構想を説明します。たとえば「小さなクラブの会員を管理したい。会費も見たい」という発言から、Plannerは利用者や業務を理解するための質問を組み立てられます。しかし、その会話に書かれた内容をすべて現在の実装事実とみなすことはできません。

設計したい機能、実際に存在するコード、データベースの現在の状態、運用上の許可は異なる情報です。利用者が口頭で要求を変更した場合は、以前の製品要件を更新する必要があります。反対に、「コードは完全に動作する」とチャットで報告されても、その特定のコードに対するテストと実行結果を確認しなければ技術的な主張は成立しません。

そのため、導入後の最初の仕事はコードを作ることではなく、文脈を構築することです。Starter内のテンプレートには仮の状態が含まれます。誤って仮の値を採用すると、他人の製品名、コード参照、受け入れ履歴が正本に混ざり、後続の全担当者に誤った前提を与えます。未確認であることを示すHOLDは、隠すべき故障ではありません。

2. 作業領域を読み取り専用で準備する

まず実アプリケーションの正確なパスを指定します。Gitブランチ、未コミット変更、ソース構成、データベースや設定の位置、既存ドキュメント、テスト方法を読み取ります。作業者が自由に書き換えられる場所と、読み取りのみ許される場所を区別します。機密情報は画面やログへむやみに表示しないでください。

v1.4でインストールした管理層には、現状を調べるための操作があります。

```
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-sdlc"
```

これらは状況を調べるための道具であり、製品の意味を自動的に発見する魔法ではありません。作業ツリーに多くの未コミット変更があるなら、誰の作業か、現在のキャンペーンに属するか、他の操作と衝突するかを記録します。Gitの更新時刻だけから担当者や承認を推測してはいけません。

環境により本番と開発用のデータベースが異なる場合は、接続先と対象スキーマを検証します。開発用に作成したサンプル行を、本番で承認された会員データとして扱ってはいけません。動いているサービスの設定を無断で変更することも禁止です。読み取り専用の探索の中で初めて、どの技術操作に追加の許可が必要かが明らかになります。

3. Plannerは製品を変える質問を優先する

Plannerが依頼者に尋ねる質問は、コードの細部を指定させるためのものではありません。製品の意味、対象ユーザー、業務上の例外、初版から除外する範囲を確定するためです。「バックエンドのサービスを何個にしますか」という質問より、「一人の会員が同時に二種類の活動へ参加することはありますか」という質問のほうが、要求の解釈を変えます。

クラブの例なら、次の点を最初に確認します。

質問	判断に関係すること
誰が会員データを登録・閲覧できるか	権限、画面、監査
同姓同名の会員をどう区別するか	IDと重複防止
会費の「支払い済み」は何を意味するか	記録と実決済の境界
退会者の履歴を保存するか	保存期間と削除規則
ネット接続がないとき使うか	オフライン動作と同期
最初の版で不要な機能は何か	キャンペーンの範囲

依頼者が答えられない技術的な細部は、Plannerや実装担当が調査する対象です。一方、「誰が会員を削除してよいか」のような業務決定は、利用者の意図や正式な所有者の判断が必要です。回答がまだない場合は、推測して完了させず、明示的な未決定として管理します。

4. 要求、Product State、コードを一緒に見る

採用の準備では、少なくとも三種類の情報を整合させます。第一は、実際に確認された製品の目的と振る舞いです。第二は、その製品の現在の状態を表すProduct Stateや関連する管理記録です。第三は、ソースコード、設定、DB、テスト結果などの技術的な根拠です。

たとえば要求に「会員を登録できる」と書いていても、アプリのAPIが存在しないなら未実装です。APIが存在していても、重複し

たIDを拒否できないなら、まだ期待した能力に達していない可能性があります。コードに書かれているという理由だけで、利用者が望んだ仕様だと決めてはいけません。

Plannerは機能ごとに現在の境界、依存するデータ、利用者の期待、受け入れ方法を整理します。技術基準点には、参照するリポジトリとブランチ、正確なコード識別子、必要な状態の識別、既知の失敗条件を含めます。更新可能なCURRENT記録に古いSHAが残っていた場合、名前がCURRENTであることより物理的な照合結果を優先します。

最初からすべての要件を百ページに展開する必要はありません。採用前に必要なのは、真実の分かる基準点と、何が分かっていないかの記録です。大きな推測を一つの巨大な文書に固定してしまうと、後で修正するたびに資料が矛盾します。権限、可変状態、履歴、キャンペーンを適切な正本へ分離してください。

5. 採用とは現実へ接続すること

正式な採用では、スターターの形式を実際のアプリケーションの事実へ結び付けます。導入契約は、この境界を説明しています。概略として、現状の読み取り、要件の確認、技術的基準点の検証、採用仕様の作成、権限や参照関係の事前点検、整合した状態の反映、再検証の順になります。

作業者が不明なコミットを適当に記入したり、テンプレートにある架空の合格記録を残したりしてはいけません。依頼者による確認が必要な決定は依頼者へ戻し、技術的な不確実性は適切な担当者が調べます。互いに矛盾する参照、確かめていない本番DB、実在しない成果物を持つ採用仕様は、表面的に整ったJSONでも採用すべきではありません。

アプリケーションがすでに管理されている場合は、新規導入の考え方を適用せず、既存プロジェクト接続の事前点検を使います。過去のキャンペーン、履歴、所有権、現在のDB基準点を維持しながら接続を評価するためです。古い管理層を削除して作り直すことは、採用ではありません。

採用の結果は「管理層が実際のプロジェクトを参照できるようになった」という状態です。そこから先に、各機能が正しく実装さ

れる工程が始まります。採用済みの記録と完成済みの機能を同じ成功表示へまとめないでください。

6. 新しい文脈で復元できるかを試す

本当に引き継ぎ可能かどうかは、以前のチャットを知らない担当者を想定したときに分かります。その担当者は、現在の要求、実装された能力、活動中のキャンペーン、変更可能な範囲、重要な証拠、次の担当者を現物から特定できるでしょうか。

reconstruct_contextで形式的な復元を試み、その結果が実際のリポジトリと矛盾しないか確認します。問題が見つかった場合は、テストを迂回するのではなく、参照の欠落、情報の古さ、権限の不一致、DB状態の不明などを分類します。ある機能に必要な情報が復元できなければ、その機能についてはHOLDとして扱い、影響しない安全な調査は続けられます。

「引き継げる」とは、別のチャットが記憶しているという意味ではありません。現時点のコード参照と明示的な採用記録から作業の正当性を再構成できることです。これは、単純な会話による開発と比較するときの重要な評価項目です。

7. 最初のキャンペーンを認可する

採用後、Plannerは価値のある最小の能力を選びます。大きな製品を一度に完成させる必要はありません。会員管理アプリなら、会員の登録と閲覧を最初の能力とし、会費と決済は次へ回すことができます。能力単位のチャーターには目的、境界、依存関係、担当者、受け入れ条件、失敗条件を記載します。

認可される前は、Plannerの考察や試作品を本番実装の権限と混同してはいけません。認可後はI3 Execution Controllerが、同じ範囲の実装、デバッグ、テスト、レビュー、再試行、復旧を一貫して担当します。一つのバグが出るたびにPlannerへ戻して別キャンペーンを作るのは、作業の連続性を損ないます。

戦略を再検討するのは、製品の目的が変わった場合、認可範囲を超える行為が必要な場合、または既存の計画を無効にする重要な

証拠が現れた場合です。通常のテスト失敗と、チャーターそのものの無効化は別です。依頼者は製品の意図と受け入れに集中し、内部の技術的な再試行を監督し続ける必要はありません。

8. 会員管理システムの初回セッション

最初の相談で利用者が「職員が会員を登録できるようにしたい」と言ったとします。Plannerは実際のコードを調べ、現状では会員テーブルは存在するが編集画面がなく、登録用APIも途中までしかないことを確認します。この結果は推測ではなく、対象のコードとDBスキーマへ参照を結んで記録します。

次に、会員に必要な項目、重複検出、アクセス権、削除の意味を利用者へ確認します。初版では氏名、連絡先、活動区分だけを扱い、実決済は行わないと決めたとします。Plannerはこの境界を記録し、会員登録・表示・入力検証を一つの能力として定義できます。

実装側には「登録の完了後に一覧で会員を確認できる」「同じ識別子を二重登録できない」「許可されない利用者は登録できない」「エラー後も既存会員は失われない」などの受け入れ条件が渡ります。どのクラスで実装するか、内部で何回修正するかは担当技術者が決めます。

完了報告には、変更したコードへの参照、使用したテスト、起動した環境、残る不確実性を含めます。利用者が実際に画面を試して「登録はできるが、確認操作が分かりにくい」と言った場合は、人間の使い勝手の問題として同じ目的の中で改善します。単にユニットテストが緑だから受け入れるのではありません。

最後に、新しい担当者へ同じリポジトリを渡す想定で再構成を確認します。どの能力が受け入れられ、会費処理がまだ対象外なのか、現在のコードの識別と共に読み取れれば、初回採用は実務的な意味を持ちます。

次へ： 第03章では日々の実行、レビュー、再試行、報告の扱いを説明します。

参照元と適用範囲

- 製品発見セッションの指示
- 既存製品の再アンカー
- 初期導入の指示
- 採用と引き継ぎ
- 既存管理層への接続
- 製品発見・要求・技術構成
- 権限モデル

この例は仮想的なクラブです。実際のコード、DB、利用者への確認がない状態で、製品の採用やキャンペーン認可が成立したとは主張しません。

03 — 日常運用と継続的な実装

> ここでは、製品の基準点が採用され、作業の権限と目的が明確になった後の一日を説明します。通常のバグ修正を何度も戦略会議へ戻さず、実際の成果と証拠を積み重ねることが中心です。権限モデルと運用プロトコルの適用範囲を確認してください。

この章で学ぶこと

1. 新しいセッションで確認するもの
2. 目的を検証可能な能力に変える
3. I3が受け持つ一連の作業
4. 外部行為に必要な許可
5. 実装報告を評価する
6. Gitと現在性
7. 一日の終了条件
8. 会員管理の運用例

1. 昨日のチャットを覚えている必要はない

新しいセッションの最初に、担当者は現在のリポジトリ、対象ブランチ、変更中のファイル、採用済みのProduct State、認可されたチャーター、活動中の実行状態を確認します。チャットの以前の報告は手掛かりですが、古いコードや異なる環境を指している可能性があります。

「前回はすべて成功したらしい」という記憶を根拠に、直ちに製品を公開してはいけません。実際の実行候補がどのコミットと設定から構成され、どのデータベースに接続し、どの検証を受けたかが必要です。別の作業者が未コミットの変更を残している場合は、所有者と干渉リスクを把握します。Gitが進んでいることだけでは、作業の担当者やキャンペーンを証明できません。

現在の情報が矛盾する場合は、影響のある変更を止めて事実を調べます。影響しない作業についてまで、無期限に全体を停止する必要はありません。重要なのは、何を安全に継続できるかを根拠に基づいて区別することです。

2. 願望を能力に変換する

依頼者の希望は、必ずしも実装単位になっていません。「会員管理を簡単にしたい」という希望には、入力画面、検索、既存データの移行、権限、操作手順などが含まれる場合があります。Plannerは目的を聞き、最初の結果を試せる能力へまとめます。

たとえば「職員が会員を登録できる」という能力なら、許可された職員が必要項目を入力し、保存結果を一覧で確認できることを定義します。重複した識別子や無効な入力を拒否すること、予期しないエラーで既存会員を失わないことも重要です。実決済や会計帳票は対象外と明記できます。

チャーターは、なぜ必要か、どこまで実装するか、どの部分に依存するか、誰が担当するか、何をもって受け入れるか、どんな結果なら不合格かを説明します。クラス名を一つ一つ指定する必要はありません。内部設計とコード変更の裁量をI3へ委ねることで、バグが見つかるたびにチャーターを書き直す混乱を防ぎます。

3. I3は途中の失敗も引き受ける

認可後のI3 Execution Controllerは、実装だけでなく、検証、デバッグ、レビュー、失敗した試行の再実行、必要な内部リファクタリング、復旧、次の候補の統合まで連続して担当します。通常のテスト失敗を「Plannerに戻して新しいGoalを作ってください」と依頼者へ押し返してはいけません。

実装者が構文エラーを出した場合、該当するコードを修正して再テストします。DBのマイグレーションが失敗した場合、元のデータを保護し、影響範囲と復旧方法を検証します。レビューで同じクラスの問題が二度以上繰り返された場合、局所修正を延々と追

加するのではなく、該当サブシステムを体系的に点検し、独立した確認まで実施する必要があります。

もちろん、I3があらゆる判断を独占するわけではありません。実装中に「会費を実際の決済事業者へ送信したい」という新要求が現れれば、元の単純な記録機能から外れます。新しいセキュリティ、費用、個人情報、外部契約の判断を必要とするため、権限者とPlannerによる再検討が適切です。失敗と範囲変更の違いを記録してください。

4. 外部行為には境界を設ける

読み取り専用のコード調査、ローカルテスト、許可済みのファイル修正と、外部公開、第三者へのデータ送信、費用を発生させるAPI呼び出し、運用DBの変更は同じ行為ではありません。後者については、目的、対象、費用・回数、復旧可能性を明確にした承認が必要です。

「GitHubに接続できた」「デプロイ用キーが環境にある」という事実は、そのキーを使って本番サービスを変更する許可ではありません。シークレットを課題の説明文やログに埋め込まず、必要な権限を最小化します。外部のAIモデルを追加するかどうかも製品上の選択であり、ArquitecturaRepoが自動的に要求するものではありません。

手元のデータを使うサンプルと本物の会員データには異なる保護が必要です。デモ用データが合成であれば、その事実を明記します。操作画面に顧客の電話番号が表示されるなら、スクリーンショット、テストログ、問題報告へ不用意に含めないようにします。

5. 完了報告は「PASS」という一語では足りない

I3から候補完成の報告が届いたら、何ができるようになったかをまず確認します。変更したファイル数やテスト件数は参考情報で

すが、機能の品質そのものではありません。特定の受け入れ条件をどの証拠で満たしたかを確認します。

検証の層	主な質問
Unit	個々の関数は期待どおりか
Integration	API、DB、画面などが一緒に動くか
DB / Currentness	テスト対象は現在のスキーマと正確な状態か
Semantic	実際の要求や例外に意味が合っているか
Physical	実環境で起動し、操作が成立するか
Perceptual / Human	利用者にとって理解しやすく、役に立つか
Release	公開対象と権限、配布物の識別が正しいか

単体テストが100個成功しても、UIが白いままなら実際の利用者の操作能力は未確認です。反対に、細かな警告が一つ残っているだけで、受け入れ可能な機能を無限に保留することも避けます。どの失敗が受け入れを妨げるかを、チャーターの目的に結び付けて判断します。

6. Gitは継続性の材料であって権限ではない

コミットは変更内容を参照する有用な手段ですが、「誰が承認したか」「何のための変更か」「現在のDBに対応しているか」を単独で答えません。リポジトリの管理記録、チャーター、テスト結果、DBの参照、成果物の識別が必要です。複数のブランチを並行して動かす場合は、同じ設定ファイルや同じデータベースへ干渉しないか評価します。

不要なクローンや重複した成果物を増やさないことも重要です。通常は一つの正本チェックアウトを利用し、並行作業の理由がある場合のみ一時的なworktreeを使います。レンダリング出力、大容量キャッシュ、テスト用動画などは、後で必要な正本証拠でない限り、一時領域や無視設定の下に置きます。

復旧可能な変更を優先してください。作業者が間違えたからといって、未確認のファイルを一括削除したり、ユーザーの変更を強制的に消したりしてはいけません。復旧操作にも対象の確認と権限が必要です。

7. 一日の終わりに答える五つの質問

作業終了時には次の五つを残すと、新しい担当者が状況をつかめます。

1. 何の製品能力が実際に増えたか。
2. その結果を証明する正確なコード・データ・成果物はどれか。
3. どの品質ゲートが成功し、どれが未検証か。
4. 未解決事項の責任者は誰で、元のキャンペーンで続けられるか。
5. 次の作業者はチャットなしでどの記録から再開できるか。

「コミットした」「AIが完了と答えた」「何時間動かした」だけでは不十分です。利用者が新しく試せる機能があるか、障害の可能性が減ったか、古い情報への依存が解消したかという結果を示します。

8. 仮想クラブでの一日の流れ

午前中、Controllerは会員登録キャンペーンの現在の状態を読み取ります。昨日の候補では登録後の画面更新が失敗し、APIとDBのテストは成功していたとします。まず候補のコード識別、ログ、再現条件を調べ、画面が古い一覧データを保持する原因を見つけます。

修正は元の登録能力の範囲に収まるため、新しいPlannerキャンペーンは不要です。実装担当は表示更新を修正し、単体・統合・ブラウザ操作の三種類を再検証します。会員データの永続性も確認し、同じ候補の技術結果と画面の証拠を記録します。

夕方に利用者が画面を試し、「登録ボタンは目立つが、保存後に何が起きたか分かりにくい」と伝えたとします。これは製品の意味に直結する人間による評価です。既存の目的の中で成功メッセージや表示を改善できるなら、I3が同じキャンペーンで修正します。利用者が「会員証を郵送する機能も追加して」と要求したら、外部の個人情報と費用を伴う別の範囲として検討します。

最後に、実際に確認した候補、残る課題、次の担当者へのルート

を正本へ保存します。翌日に別のチャットが開始されても、一から事情を説明し直す必要がない状態が日常運用の目標です。

次へ： 第04章では、利用者が変更をどのように伝え、優先順位を付けるかを説明します。

参照元と適用範囲

- 権限モデル
- 運用モデル
- 実装と品質
- 製品発見と計画
- 公開v1.4の使用境界

この章の運用例は架空のものであり、特定のアプリケーションのテスト成功や公開を主張するものではありません。

04 — 変更を依頼する方法

利用者は、データベースのテーブル名やクラス構成を指定しなくても製品を方向付けられます。この章では、要求の意味を明確にし、必要な判断を行い、通常の不具合修正と新しい製品の要求を区別する方法を扱います。

この章で学ぶこと

1. 実装方式より結果を伝える
2. 短い変更依頼の書き方
3. 利用者と技術担当の責任
4. 意見を変更した場合の履歴
5. 品質と使いやすさの要求
6. 優先順位と範囲
7. 判断が必要な質問
8. 実例と良くない依頼の改善

1. 最初に「何を可能にしたいか」を伝える

「会員一覧が重いので、データベースのインデックスを追加してください」という指示より、「会員が増えても検索結果が数秒以内に表示され、職員が作業を続けられるようにしたい」という依頼のほうが、製品の結果を明確にします。インデックスは一つの実装候補にすぎず、真の原因が画面側の無駄な読み込みなら最適な手段ではありません。

利用者が解決したい問題、誰が困っているか、どの状況で発生するか、どんな結果なら役に立つかを伝えてください。可能であれば具体例も添えます。たとえば「400人の会員から姓で探したい」「同姓同名が二人いる」「退会した人を誤って再登録したくない」などです。

技術担当は、その結果を実現するための設計、実装、テストを検討します。利用者がプログラミング経験を持っている場合は技術案を提案してもかまいません。ただし「この方法でなければならない」という契約にする前に、代替方法と費用・安全性を確認してください。

2. 短い変更依頼のひな型

必要な情報は一枚の簡単な説明から始められます。

項目	記入例
目的	会員を名前と識別番号で探せるようにする
利用者	受付の職員
現在の困りごと	一覧を手作業でスクロールするため時間がかかる
期待する結果	入力すると対応する会員が見つかり、複数候補を識別できる
重要な条件	許可のない利用者へ連絡先を表示しない
初版では不要なこと	複雑な統計、外部サービスとの自動同期
受け入れの例	二人の同姓同名会員が正しく区別される
未決定	検索結果に退会済み会員を含めるか

これは実装チャーターそのものではなく、製品の意図を伝える入口です。Plannerは現物のコードと要求を調べ、未決定事項を確認し、適切な能力単位へ整理します。依頼文に細かな技術指示を詰め込み、後から実装者が一字も変えられないようにする必要はありません。

「全部をきれいにして」と依頼する場合も、何を意味するかを追加すると有用です。たとえば「会員登録までの操作を三段階以内にする」「誤入力時に既存のフォームを失わない」「電話画面で

もボタンが押しやすい」といった、確認可能な目標にします。操作感に関する要求は、コードの内部テストだけで判断しないことが重要です。

3. 利用者が決めるものと委任するもの

利用者が担当するのは製品の意図です。どの利用者がどんな操作を必要とし、何を最優先し、何に費用を払えるか、どの外部公開やデータ取り扱いを認めるかを決めます。社内規則や法律について判断権限を持たない場合は、該当する正式な責任者へ確認します。

内部の技術担当は、どう実現するかを引き受けます。Gitの衝突解消、技術構成、検証用環境、テスト、データ移行、ログや復旧の設計を、毎回利用者へ細分化して尋ねる必要はありません。重大な制約や費用が製品判断へ影響するときは、選択肢とその結果を分かる形で説明します。

I2 Plannerは製品の問題を実行可能な能力へ整理し、I3 Execution Controllerは認可された範囲で継続して実装します。I4は限定された実装や独立レビューを担えます。複数製品の統合と共有資源の判断が必要な場合はI1を設けることがあります。役割は責任の区分であり、必ず四つの個別AI会話を必要とするものではありません。

4. 意見を変えても履歴を壊さない

最初は「退会者を一覧から完全に削除する」と要求し、後から「以前の会費履歴は残したい」と考え直す場合があります。新しい明示的な製品決定は、以前の要求を置き換えることができます。しかし、過去の判断が一度存在した事実や、すでにその判断に基づいて作成したデータを消す必要はありません。

変更の前後を分かる形で記録します。いつ、どの範囲の要求が変化し、既存コードやDB、バックアップ、公開済みの操作にどんな影響があるかを調べます。実装済みの削除処理が、変更後の要求に反しているなら、必要な移行と安全な復旧の計画が必要で

す。

Product Stateと現在の要求は正本に反映し、古い設計案は履歴として参照します。二つの文書が共に「現在の決定」と主張すると、新しい担当者が誤った判断をします。たくさんの要約ファイルを増やすより、一つの明確な現在状態と参照可能な履歴を維持してください。

5. 品質に対する要求を明確にする

「動いているが見た目が弱い」「動作が遅い」「エラー時の説明が分かりにくい」は、正当な製品フィードバックです。利用者は、その体験を具体的な場面に結び付けて伝えられます。機能が存在するだけでは、使える製品としての受け入れが完成するとは限りません。

たとえばボタンが表示されない場合は、対象の画面、操作手順、端末・ブラウザの条件、スクリーンショットを提供します。これはデザインの好みではなく物理的な不具合です。表示はされているが目立たない場合は、人間による知覚評価の問題として扱います。実装側は両者の検証方法を分けます。

性能の要求も、試験条件なしでは曖昧になります。「会員500件の一覧が通常の職員端末で3秒以内」「低速な回線で操作不能にならない」といった条件が有用です。ただし数値は例示であり、実際の利用者や運用環境に合わせて決めてください。どの機能にも同じ測定値を強制する必要はありません。

6. 優先順位は実際の価値で決める

要望が十件集まったとき、十個のキャンペーンを同時に立ち上げることが最適とは限りません。今必要な利用者の操作、依存する機能、重大な欠陥、コスト、実施による干渉を考慮して順序を決めます。受付業務ができないなら、装飾的なレポートより会員検索の改善が先かもしれません。

まず一つの利用者経路を完成させ、それを確かめてから次へ進む方が効率的な場合があります。実装中のバグは元のキャンペーン

内で処理します。Plannerが関与するのは新しい能力の戦略、認可範囲の変化、重要な新証拠、最終的な受け入れの判断です。

優先順位の変更も、今あるキャンペーンの成果や依存関係を確認して行います。以前の作業を無言で無効化すると、中途半端なマイグレーションやテスト用状態が残る可能性があります。中断する場合は、安全な停止点、再開に必要な情報、次の責任者を記録します。

7. 判断が本当に必要なとき

「使うDBをPostgreSQLにしますか、それともSQLiteにしますか」という問いは、一般には技術担当が要件から提案できます。一方、「会員の個人情報を外部のサービスへ送信してよいか」「本番の会費履歴を削除してよいか」「有料APIの費用を引き受けるか」は、適切な権限者の承認を必要とします。

判断が必要になった場合は、選択肢、具体的な利点と不利益、影響を受けるデータ、元に戻せるか、確認していない点をまとめます。利用者に毎回長いシステム設計会議を求める必要はありませんが、認可のない外部行為を作業者の判断だけで実行することもできません。

未確定の事項があっても、無関係なローカルテストやコード調査は継続できます。外部公開に関する承認がまだなくても、公開を伴わない画面の修正までは安全に進められる場合があります。停止するのは不確実性に依存する操作です。

8. 良い依頼の具体例

次の五つは、結果と境界を分けた依頼です。

1. 「職員が登録済み会員を名字と識別番号で探せるようにしたい。検索結果の連絡先は権限のある職員だけに表示してほしい。」
2. 「新規会員を保存した後、完了したことが分かる画面にしたい。入力エラーでは記入した内容を保持してほしい。」

3. 「会費が未納の会員を見たい。ただしこの段階では実際のオンライン決済を実行しない。」
4. 「スマートフォンでも登録画面を操作しやすくしたい。どのボタンが重要か職員と確認したい。」
5. 「既存の会員データを失わずに新しい形式へ移行したい。移行前に復旧可能なバックアップを用意したい。」

反対に「もっとプロらしく」「AIで全部自動化」「全部のエラーをなくす」とだけ書いても、完了を判断できません。Plannerは具体的な利用場面を聞き、検証可能な単位に分けます。利用者の要求は技術命令である必要はありませんが、解決したい問題が分かることは大切です。

技術的に完成した候補でも利用者が気に入らなければ、使い勝手や意味に関する具体的な不一致を伝えてください。元の能力に含まれる修正なら同じキャンペーンで改善できます。要求を大幅に広げる変更なら、別の製品判断として扱います。この区別が、継続的な実装と柔軟な製品設計を両立させます。

次へ： 第05章では、問題が発生したときの調査、データ保護、復旧を扱います。

参照元と適用範囲

- 製品発見と要求の整理
- 役割と権限
- 引き継ぎと採用
- 公開利用者の案内
- 日本語のFAQ

例示した画面、データ量、応答時間は実際のクラブで測定された値ではありません。導入先で要求を確認してください。

05 — エラーの診断と安全な復旧

ArquitecturaRepo · 完全ユーザーマニュアル · v1.4版 · 日本語

> エラーが発生したときに最初に守るべきものは、実際のコード、データ、現在の状態を再構成するための証拠です。緑色のテスト表示を取り戻すために記録を削除してはいけません。実行可能な検証と既存の管理層への接続も参照してください。

この章で学ぶこと

1. 原因を調べる前に保護するもの
2. 環境や接続の失敗
3. 管理層の検証失敗
4. Git操作とデータ消失
5. 繰り返す失敗への対応
6. プライバシー、セキュリティ、権利
7. 障害を終了するための証拠
8. 三つの具体的な復旧例

1. 原因より先に証拠を保護する

問題が起きた場合は、何が失敗したか、いつ起きたか、どの環境・設定・入力・コードの版で発生したかを記録します。再現できる操作順序、エラーメッセージ、関係するファイル、可能なら安全なログを残します。個人情報やアクセスキーを含むログは、そのまま公開リポジトリへ添付してはいけません。

実行中のキャンペーン、未コミットのファイル、DBの現在性、成果物のハッシュと由来を確認します。削除や上書きを含む操作は、正確な対象を把握するまで保留します。誤った変更を「なかったこと」にすると、後で真の原因を調べるための情報が失われます。

障害の影響を分類してください。単にローカル開発環境が起動しないのか、一つのテストが失敗したのか、利用者が本番で操作できないのか、あるいはデータの完全性が危険にさらされているのかによって優先順位が異なります。未確認の本番状態があるなら、その環境へ書き込む前に承認と復旧可能性を確認します。

安全な診断は、原則として読み取りから始まります。再現のために大量の会員データを作るなら、開発環境の合成データを使用します。利用者のデータをコピーする必要がある場合は、許可、最小化、保存先、削除方針を検討します。

2. 環境とアカウントの問題

PythonやGitが見つからない、依存関係を読み込めない、シェルの作業ディレクトリが違うなどは、製品の要求が間違っていることを意味しません。現在の端末で使用しているインタープリタ、PATH、必要なライブラリの版を調べます。

```
& "python" --version
& "git" status --short
Get-Location
```

必要な検証用ライブラリは、インストールされた管理層のrequirements-validation.txtから導入します。パッケージが不足しているだけなら、Product Stateの承認を偽装してテストを通す必要はありません。接続認証の失敗なら、権限と対象アカウントを確認し、秘密情報をチャットへ貼り付けずに復旧します。

GitHubや外部のエージェントが一時的に利用できない場合でも、ローカルにあるコードと文書の読み取りは継続できる場合があります。ただし、外部サービス上で公開・統合済みであることをローカルログだけから推測してはいけません。確認できない状態は未確認のまま扱います。

3. 管理層の検証が失敗する場合

検証器が不足ファイル、スキーマ違反、現在状態の矛盾、無効な参照を報告した場合、まずどの条件に違反したかを確認します。HOLD / UNTRUSTED_CONTEXTは初回の採用前であれば正常な保護状態です。問題を分類せずに、すべてをREADYに書き換えるのは誤りです。

検証器そのものにバグがある場合もあります。そんなときは、どの実例が誤って拒否されたか、プロトコルの意図と検証条件が一致しているかを確認します。検証器修正は、既存キャンペーンの技術的な再作業の範囲で処理できます。実装が失敗するたびにPlannerへ戻る必要はありません。

一方、参照先のDBが本番から開発用に切り替わっている、古いコードを現在と主張している、消えた成果物を受け入れ証拠にしている場合は、単なる形式修正では解決しません。必要な正確な情報を確保し、互いに整合した状態を再構築します。証拠の穴を架空の成功記録で埋めることは厳禁です。

4. Gitは安全な万能取り消しではない

作業者のブランチに未コミット変更がある場合、それが不要であるとは限りません。別の担当者が作成中の修正、調査用の証拠、まだ統合されていない成果物かもしれません。強制的なリセット、ワークツリーの削除、広いディレクトリの一括消去を行う前に、正確な対象と所有者を確認します。

復旧方法には、差分を保持してバグを直す、変更を別の安全な場所へ退避する、レビュー済みの修正コミットを作る、データを別途復元するなどの選択肢があります。どれを選ぶかは、現状と権限によって異なります。Gitはデータベースの全履歴や外部公開操作を元に戻してくれるわけではありません。

特に会員管理のDBでテーブルを削除した場合、コードを以前のコミットへ戻しても失われた会員が復元されるとは限りません。移行とロールバックは、コードだけでなくスキーマ、データ、バックアップの整合性を考える必要があります。復旧操作を本番で試す前に、承認と利用可能な復元点を確認します。

5. 同じ失敗を何度も繰り返さない

一度のテスト失敗なら、その原因を修正し再検証します。しかし同じサブシステムで失敗と差し戻しが二度、三度と続く場合は、局所修正の積み重ねが問題を隠している可能性があります。I3は対象サブシステム、依存する設定、起動方法、例外条件、データの流れを体系的に点検する必要があります。

たとえば登録後の画面更新が何度も壊れるなら、単に最後の条件分岐を修正するだけでなく、API応答、クライアントのキャッシュ、イベント更新、DBの一貫性、テストの再現性を検証します。独立したレビュアーが実装者の前提を検討することも有用です。

このような技術的な強化は、基本的には既存の認可されたキャンペーンに属します。「再試行番号R8」「新しい回復チャーター」を際限なく増やしても、利用者に見える能力は増えません。戦略的に新しい証拠がない限り、Plannerの次の仕事は技術的な試行ごとの微調整ではなく、統合された結果の監査です。

6. セキュリティ、個人情報、ライセンス

会員管理アプリのログには、名前、住所、電話番号、支払情報が含まれる場合があります。障害を調査するために必要な範囲のみ扱い、共有する記録から不要な個人情報を除きます。トークンやAPIキーが漏えいした可能性があれば、認可された運用責任者と連携して適切に無効化・再発行します。

外部の画像、フォント、ライブラリ、動画を使う場合は、利用条件を確認します。検証レポートに資料がないことと、法的に使用が禁止されていることを混同しないでください。必要な使用許可と出典が満たされているかを現実的に確認し、不明で重要な場合は明示的な判断を求めます。

ArquitecturaRepoのフィードバック機構は、ローカルでメタデータのみを扱う方針と、外部送信を別に扱います。定期実行のワークフローを無効にしても、ローカル生成が無効になったとは限りません。実際の設定とプライバシーポリシーを確認し、第三者へデータを送信する場合は明示的な許可を得ます。

7. 復旧の終了条件

復旧が成功したと報告するには、原因が説明され、問題が解消され、同じ条件で再発しないことを合理的に検証できている必要があります。関連するゲートを別々に記録し、対象のコード、設定、DB、実行環境を明示します。バックアップから復元した場合は、復元したデータの期間と整合性も確認します。

一つのテストが成功したことを全体の完了と混同しないでください。物理的な起動、利用者の操作、セキュリティ上の境界、公開準備が必要なら、それぞれの証拠が必要です。反対に、検証の目的に関係しない小さな警告だけで復旧を無期限に止めるべきでもありません。

最後の記録には、問題の要約、原因、修正された範囲、具体的な検証、残る危険、次の責任者を書きます。新しいセッションでも、同じ障害を最初から調べずにすむ状態を作ることが重要です。

8. 三つの復旧例

例A：インストール後のHOLD

新しいクラブアプリに管理層を導入し、reconstruct_contextがHOLDを返します。担当者が確認すると、Product Stateに実際のコードと要件への参照がありません。これはインストーラの故障と断定できません。正しい対応は、新規プロジェクトの発見と採用を行い、確認された要求と技術基準点を結び付けることです。

誤った対応は、テンプレート内のREADYフラグを手動で変更し、仮のコミットSHAを入れることです。その状態でPlannerが実装を指示すると、存在しないコードを前提にする危険があります。まず現状を調べ、分からない事実を明示したまま、必要な確認を進めます。

例B：画面は成功するが保存されない

会員登録画面が完了メッセージを表示しても、画面を更新すると新しい会員が消える場合があります。画面だけをテストした結果では実装の成功を証明できません。APIの応答、保存処理、DBのトランザクション、画面の更新を一つの利用者経路として調べます。

保存先がテスト用DBだったことが分かった場合、現在の環境を正しく識別してから修正します。既存の会員を損なわずに統合テストを行い、アプリを起動して登録・閲覧を再確認します。元の会員登録能力の範囲に含まれるバグなら、I3が同じキャンペーンで処理します。

例C：移行後に会費履歴が欠ける

新しい会費形式への移行で過去の記録が欠けた場合は、まず破損が広がる書き込みを止めます。バックアップ、移行ログ、DBの現在状態を読み取り、失われた範囲を調べます。コードを戻すだけでは会費の履歴は復元されません。

バックアップが存在し、復元の権限がある場合は、隔離した環境で復元方法とデータの整合性を検証します。本番で復元する際には、影響を受ける利用者、作業時間、逆戻り可能性を確認します。復旧後は会費の残高だけでなく、個別履歴と関連会員の対応が一致するかをチェックします。

次へ： 第06章では、機能が本当に完成したかを判定するための受け入れと終了を扱います。

参照元と適用範囲

- 実行可能な検証
- 既存管理層の接続
- プライバシーとフィードバック
- 権限モデル
- 公開スターターのサポート境界

ここに示した障害は説明用であり、実際の製品で起きたことや修正済みであることを主張するものではありません。

06 — 能力の受け入れと作業の終了

ArquitecturaRepo · 完全ユーザーマニュアル · v1.4版 · 日本語

> 技術的な作業の終了と製品能力の受け入れは同じではありません。この章は、何を実際に確認すれば「完成」と言えるのか、何を同じキャンペーンへ差し戻すのか、いつ新しい判断が必要なのかを整理します。

この章で学ぶこと

1. 能力とは利用者にとっての結果
2. 終了の五つの要素
3. 一つの候補と一連の証拠
4. 不足する証拠の扱い
5. 受け入れ、差し戻し、目的変更
6. 繰り返す確認と停止条件
7. 受け入れ記録を正本に残す
8. 会員登録の判断例

1. 「機能を作った」だけでは能力にならない

利用者が「職員が会員を登録できるようにしたい」と希望した場合、実装者が登録用の関数を書いただけでは能力の受け入れは成立しません。正しい権限を持つ職員が画面から入力し、保存された会員を確認でき、失敗時にもデータが破壊されないことなど、製品として意味のある結果が必要です。

内部の成果には価値があります。ソースコードが整理され、テストが増え、重複処理が削減されることは保守性を改善します。しかし受け入れ判断では、それらがどの利用者経路や危険の低減を支えているかを確認します。「コミットが30個」「エージェントが20時間作業した」という数字を、製品能力が増えた証拠として

使わないでください。

機能は小さくてもかまいません。会員登録だけを完了し、会費計算やオンライン決済を後に回すことは合理的です。むしろ、最初からすべてを完成したように見せかけると、何が現実に使用でき、何が未実装なのか利用者が判断できません。

2. 終了を五つの要素で考える

最終的な評価では、少なくとも五つの観点をそろえます。

観点	確認事項
目的	認可された要求と結果が一致しているか
候補	どのコード、設定、DB、成果物の組み合わせか
証拠	適切な検証が実行されたか
人間による判断	必要な使いやすさや知覚品質を確認したか
継続性	受け入れ結果と残りの問題を正本から復元できるか

構文テスト、統合テスト、DB/currentness、意味、物理的な動作、人間の評価、公開の準備は、別々のゲートです。要求がデータの保存に関係する場合はDBの確認を省略できません。実際の画面を使う製品なら、ユニットテストだけで画面の受け入れを完了したことにもできません。

同時に、あらゆるプロジェクトへ同一の大規模な評価儀式を強制するべきでもありません。小さなCLIツールに、人間の視覚的デザイン審査が必要とは限りません。必要なゲートは、能力の目的とリスクに比例して定義します。重要なゲートが未検証なら、その部分を未確認として扱います。

3. 正確な候補を確認する

同じ機能を何度も修正すると、過去のテストが現在のコードに対して有効か分からなくなります。受け入れ対象には、少なくともコード版、必要な設定、データとスキーマの識別、実行環境、関連する成果物を結び付けます。

「昨日のV3では画面が動いた」という報告だけでは、今日のV4を受け入れることはできません。V4で変えた場所が画面以外であっても、依存関係が変われば物理的な動作が異なる可能性があります。再実行が必要な検証範囲を明確にし、古い結果を無断で引き継がないでください。

ソースコードのハッシュは重要ですが、それだけで本番のデータベース、外部設定、利用者の権限を固定できません。必要な情報を一つの候補に関連付けて、受け入れ判断の対象を特定します。成果物を再生成した場合も、古い画像や動画のハッシュを新しい候補に流用しないようにします。

4. 証拠が足りないときは作業を区別する

ある候補のユニットテストは合格しているが、実環境で会員登録ができるか未確認という場合、正しい判断は「ユニット検証済み、物理的動作未確認」です。未確認の部分をPASSへ変更してはいけません。

不足している証拠が元の能力に含まれるなら、I3は同じキャンペーンでテストや評価を追加できます。必要な検査のために新キャンペーン、新Goal、新しいPlannerの戦略パスを必ず作る必要はありません。差し戻しでは、どの条件が満たされていないかを具体的にまとめます。

逆に、ユーザーが突然実決済を要求した場合は、単なるテスト追加ではありません。元の会費記録に比べ、外部資金、契約、セキュリティ、復旧方法の意味が変わります。認可範囲の外へ進む前に、必要な製品判断と新しい境界を確認します。

証拠が見つからないことと、必ず不具合があることも別です。テスト結果が保存されていない場合は、そのテストが失敗したと断定できません。未実施・記録欠落・実際の不合格を分け、必要な検証を合理的な範囲で実施します。

5. 三つの最終判断

一つ目は「受け入れ」です。定義された能力を利用者が実際に使え、必要な品質・リスク条件を満たし、候補と証拠が再構成可能なときに適用します。残る軽微な改善は後続の作業へ回すこともできますが、受け入れ条件に関係する欠陥を隠してはいけません。

二つ目は「同じ範囲で修正」です。機能の目的は変わらず、具体的なバグ、統合失敗、性能不良、画面の使いにくさなどが残る場合です。I3にまとめた失敗条件を返し、同じキャンペーンで修正、再検証、独立レビューを進めます。差し戻しのたびに新しい戦略計画を起こす必要はありません。

三つ目は「目的の再定義」です。利用者の要求そのものが変わる、技術的制約が当初の境界を無効にする、重大な新事実により受け入れ条件が適切でなくなる場合です。そのときはI2 Plannerや必要な権限者が、変更した目的と依存関係を整理します。

6. 終わりのない検査を避ける

品質は重要ですが、受け入れ可能な能力を無期限に保留するのは別の問題です。受け入れに関係する失敗条件がなくなり、重要な品質ゲートが満たされ、残るリスクを権限者が理解しているなら、終了できます。

反対に、同じ種類の不具合が何度も繰り返されるのに、細かな修正を毎回「最終版」と呼ぶべきでもありません。そのサブシステムを深く検証して原因を解消し、独立レビューで確認します。正しい停止条件は、資料やテストの数ではなく、能力のリスクと受け入れ条件で決まります。

「すべての可能な環境で完全に安全」という証明は、通常は現実的ではありません。試したOS、端末、入力条件、データ量を正確に記録し、試していない条件を区別してください。必要なら公開を段階的に行い、復旧可能性を維持します。

7. 受け入れ記録は新しいチャットでも読めるようにする

最終判断をチャットの一言にだけ残してはいけません。正本には、対象能力、認可されたチャーター、採用した候補の識別、検証の証拠、判断者、残るリスク、次のルートを保存します。どこが現在の状態で、どこが履歴なのか明確にします。

新しい担当者が、既に受け入れられた会員登録機能を誤って最初から作り直したら、運用の継続性は保たれていません。逆に未検証の会費機能を、過去のキャンペーンのPASSから推測して公開するのも危険です。各能力の範囲と現在性を分離することで、後続作業が安全になります。

完成した機能に新しい要求を追加する場合も、元の受け入れ記録を書き換えて過去をなかったことにしません。受け入れ済みの版と、その後の変更要求を追跡可能にすることで、問題発生時にどの段階から修正すべきかを判断できます。

8. 会員登録の判断例

仮想クラブで、会員登録のフォームが完成したとします。ユニットテストは成功し、DBへの保存も確認されています。しかし受付担当が実際に使うと、保存ボタンを二回押したときに同じ会員が重複登録される問題が見つかりました。

この場合、製品能力はまだ受け入れられません。I3は元のキャンペーンで二重送信対策と重複防止を修正し、同じ入力と別の入力を使った統合テスト、実際のフォーム操作、DBの一意性を検証します。すべてを新キャンペーンへ移し替える必要はありません。

次の候補では重複が防止され、登録結果が明確に表示され、許可のない利用者は登録できないと確認できました。利用者も実際の操作を試し、初版に必要な流れを承認しました。証拠と版の識別が残っていれば、この能力を受け入れて次の会費記録へ進めます。

ただし、オンライン決済までは受け入れていません。会員登録が

完成したからといって、支払いを実行できる、金銭を安全に扱えると主張するのは誤りです。次の能力は、別の目的、依存関係、受け入れ基準を持つ必要があります。

次へ： 第07章では、この区別を最初の発見から二つの能力の受け入れまで一つの例で追います。

参照元と適用範囲

- 権限モデル
- 検証契約
- 導入と引き継ぎ
- 製品発見と要求
- 公開利用者向けの境界

ここで挙げた会員登録は学習用の仮想例です。検証結果は実在する製品に対する保証ではありません。

07 — 実践例：小さなクラブ管理アプリを構築する

> 最後の章では架空の小規模クラブを例に、目的の整理からスターターの導入、製品の採用、会員登録の実装、会費記録への展開、最終的な引き継ぎまでを一続きで追います。これは説明用のシナリオであり、実在するアプリケーションの作成や検証が完了したという報告ではありません。

この章で学ぶこと

1. 製品の目的と対象者
2. 要件を変える質問
3. スターターの導入
4. 最初の能力の設計
5. 実装担当への引き継ぎ
6. テストと人間による確認
7. 二つ目の能力と依存関係
8. 障害、修正、継続性
9. 最後の復元演習

1. 架空のクラブの問題を定義する

地域の小さなスポーツクラブを考えます。受付の職員は紙や表計算ソフトで会員を管理しており、名前の検索、活動区分の確認、月ごとの会費記録に時間がかかっています。クラブの責任者は「登録、一覧、会費の状況を一つの画面で確認したい」と希望しています。

ここで最初から、オンライン決済、顔認識、AIによる会員予測、外部の会計システム連携を追加する必要はありません。初版では

会員を登録・検索でき、会費を記録として扱える小さなアプリケーションを目標とします。実決済はしないという境界を明示します。

関係者は、受付職員、クラブ責任者、会員自身です。誰がデータを閲覧し、修正し、削除する権限を持つかは製品判断です。保存する情報に名前や連絡先が含まれるため、プライバシーと権限が重要です。永続データを持つという理由から、導入時のプロファイルはSTATEFULを検討できます。

「会員管理アプリが完成した」という結果はまだ存在しません。ここであるのは、依頼者の希望と仮の要件だけです。次の段階で、実際の既存コードと利用者の意思を確認します。

2. 仕様を決める質問をする

Plannerは、会員の識別方法、活動区分、退会、会費の意味、利用者権限を確認します。たとえば「同じ名字の会員が複数いるときどう区別するか」という質問がなければ、一覧画面だけを作っても誤った会員を編集してしまう可能性があります。

もう一つ重要なのは、「会費を支払った」と表示する条件です。現金を受付が受け取った事実を職員が記録するだけなのか、オンライン決済プロバイダへ送金するのかでは、必要な契約、技術的保証、セキュリティが異なります。今回は前者だけを対象にします。

要件	初版での確認例
会員の識別	一意の識別番号を使用する
登録項目	氏名、連絡先、活動区分
重複の扱い	同じ識別番号を二重に登録しない
権限	受付職員は登録できるが、一般閲覧者はできない
退会	履歴を残すか正式な運用者に確認
会費	支払記録の入力に限定し、決済実行はしない
初版の除外	外部同期、自動請求、複雑な統計

これらはシナリオを進めるための仮の判断です。現実のクラブへ導入するなら、実際の担当者に確認しなければなりません。製品

の意図を決めるのはCLIENTであり、技術担当が便宜上勝手に確定するものではありません。

3. スターターを正しい場所へ導入する

開発者はArquitecturaRepoを入手し、アプリケーション本体のリポジトリとは分離します。すでに管理層があるなら既存プロジェクトの接続手順を確認します。新しいプロジェクトでは、対象を点検してからスターターのルートで次の例に相当するコマンドを実行します。

```
& "python" "tools\scaffold_project.py" --target "..
\club-app"    --profile    "STATEFUL"    --product-
id "CLUB_MANAGER" --objective "Club member and dues records
-language "en" --no-feedback-workflow
```

その後、対象のclub-appに生成された .agentic-sdlc/ を確認し、検証用依存関係を導入して次の調査を実行します。

```
& "python" ".agentic-sdlc\tools\project_status.py" -
-control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" -
-project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" -
-root ".agentic-sdlc"
```

HOLD / UNTRUSTED_CONTEXTが返されても正常な場合があります。この時点では実際のクラブのコード基準、DB状態、採用済み要求がまだ確認されていないためです。生成された管理ファイルが存在するという事実と、実製品が採用済みであるという主張を区別します。

4. 採用のための技術調査

仮に対象アプリケーションへ、会員テーブルの試作と一覧を表示するコードがすでに存在するとします。Plannerはそれを読み取

り専用で調査し、どの機能が本当に実装済みか、どのテストが実行可能か、どこにDBとそのマイグレーションがあるかを確認します。

チャットに「登録機能はほぼ完成」と書かれていても、その表現をコードの現在性へ置き換えません。実際のソース参照、設定、スキーマ、必要なデータの基準点を確認します。テストが実行されていなければ未検証とします。担当者や以前の作業の権限が不明なら、確かめるまで書き込みを行いません。

次に、利用者が確認した要求と技術的な基準点を管理層の適切な正本へ結び付けます。採用の仕組みについては移管・採用契約を参照してください。形式が整ったJSONが存在するだけでは、現実と一致していることは保証されません。

最初の採用で必要なのは、実製品についての正確な最低限の情報です。いきなり将来の機能すべてを実装する必要はありません。後続の作業者が現状を理解できる、信頼できる出発点を作ります。

5. 最初の能力：会員の登録と一覧

Plannerは最初の実装能力を「許可された職員が会員を登録し、一覧で確認できる」と定義します。保存された会員が再起動後も残ること、識別番号が重複しないこと、権限のない利用者が登録できないことを含めます。オンライン決済と会費の記録はこのキャンペーンに含めません。

依存関係としてDBの保存先、会員のスキーマ、アクセス権、画面とAPIの接続が必要です。受け入れは、単に登録関数のユニットテストだけでなく、登録画面から保存し、一覧で再確認する利用者経路で評価します。

この目的、範囲、依存関係、担当者、受け入れ条件、失敗条件を一つの能力レベルのチャーターへまとめます。内部のファイル名や修正順序を一行ずつ指定する必要はありません。実装の技術的な自由度を残すことが、途中でバグが見つかったときの修正を容易にします。

認可されたら、I3 Execution Controllerが実装、統合、レビュー、修正、再試行、候補作成を引き受けます。CLIENTが一つ一つのテ

スト失敗を承認し直す必要はありません。ただし、製品の要求や外部行為の権限が変わる場合は、新たな判断が必要です。

6. 第一候補で起きる二つの問題

仮想的な最初の候補では、APIの登録処理は成功しました。しかし、保存直後に画面が更新されず、職員が登録結果を確認できませんでした。これは単体テストが緑でも、物理的な操作が不十分である例です。

I3はコードの現在の版を確認し、API応答、画面の状態更新、一覧の読み込みを調査します。修正後に統合テストと実際の画面操作を再実施します。元の会員登録能力の範囲内なので、新しいキャンペーンは不要です。

第二の問題として、同じ登録ボタンを二回押すと会員が重複して作成されました。画面だけの二重クリック防止で十分か、それともDB側にも一意制約が必要かを実装側が判断し、異常な再送にも耐えられることを検証します。既存会員が消えたり、他の職員が同時に操作して不整合を生む場合は、さらに原因を調査します。

修正候補ができたら、何のコード版・DB・設定でテストしたかを記録します。ユニット、統合、DB/currentness、意味、物理的な動作、人間による操作、公開準備という各検証層を区別します。試していない項目を合格と書くことはできません。

7. 利用者の確認と受け入れ

技術的な検証が成功した後、受付職員が実際の操作を試します。「入力して保存できるが、成功したことが分からない」という評価が出たとします。成功メッセージと画面表示の改善は、元の能力を実際に使えるものにするための作業です。

I3は同じキャンペーンで改善し、再確認を求めます。職員が必要な操作を完了でき、権限とデータ保存の確認が取れたら、CLIENTは最初の能力を受け入れられます。受け入れ記録には候補の識別、検証結果、残る制限、判断者を残します。

「会員登録と一覧が使える」という受け入れは、「会費が正しく計算される」「決済が安全に行われる」という保証ではありません。能力の境界を明示することで、次の作業の目的を正しく設定できます。

8. 二つ目の能力：会費を記録する

次にPlannerは「職員が会員の会費支払いを手動で記録し、未納状況を確認できる」という能力を検討します。これは第一の会員識別機能へ依存します。会員のIDが安定していなければ、誤った人へ支払い履歴が結び付く可能性があります。

利用者に、対象期間、金額の扱い、返金や取り消しの意味、退会者の記録、誰が修正できるかを確認します。実決済は含めないという境界を維持します。表現上「支払い済み」と表示されても、それは職員による入金記録であり、外部銀行や決済事業者の確定結果ではないことを明示します。

この能力のテストでは、同じ月の重複記録、誤った会員ID、取り消し、日付境界、権限、再起動後のデータ保持などを確認します。第一の能力で成功したテストを流用するだけでは、会費固有の意味を証明できません。

ここまで進んでも、クラブ管理の将来のすべての機能が完成したわけではありません。活動予約や分析レポートなどは、実際の需要が確認されたときに追加できます。小さな能力を確実に受け入れて積み重ねることが、誇大な完成宣言より持続的です。

9. 最後の復元演習

次の日に、別の担当者へリポジトリだけを渡すと想像してください。その担当者が確認すべき項目は次のとおりです。

1. 製品の現在の目的と、CLIENTが承認した範囲は何か。
2. 会員登録と一覧について、どの候補が受け入れられたか。
3. 会費記録の能力は未着手、進行中、受け入れ済みのどの状態か。
4. 現在のコード、DB、設定、検証結果はどこにあるか。

5. 未解決の問題と、その修正を担当するI3または関係者は誰か。
6. 実決済や外部公開など、まだ認可されていない行為は何か。

これらをチャットの全文を読むことなく説明できれば、継続性の基盤が実際に機能しています。答えが分からない場合は、参照元や正本が不十分かもしれません。未確認を隠さず、必要な事実を復元することで次の作業者が安全に再開できます。

最後に、比較の観点も忘れないでください。ArquitecturaRepoが単純なコーディングチャットより常に速いとは限りません。判断すべき指標は、実際に使える機能、変更時の混乱の少なさ、失敗からの復旧、新しい担当者への引き継ぎ、利用者が管理しなくてよい技術的負担です。長期間続く製品でその価値が示されるかを、実際のプロジェクトで測定してください。

完了： これで全八章です。簡易版は日本語ミニガイド、よくある質問は日本語FAQから参照できます。実際の運用では、対象バージョンの公開導入ガイドと現物のコードを優先してください。

参照元と適用範囲

- プロファイル定義
- 新規導入スクリプト
- 導入と移管
- 製品発見と要求の整理
- 権限モデル
- 実行可能な検証
- 公開開始とサポート境界

この章は作業順序の説明であり、架空のクラブのプログラム、データ、テスト結果、実際の公開物を提供したものではありません。